

StockRoom

Hotel Raw Material Management — Developer Problem Statement

Build-a-thon brief · 6 hours · Fresher Engineering Track

You are given a real product problem and a fixed window of six hours to ship something that runs. We are looking at how you think, how you scope under pressure, and how you keep a system's records correct when stock is moving through it in several directions at once.

Read the whole brief before you write any code. The hard part of this problem is not any single screen — it is making the recorded stock match the real stock as deliveries, issues, consumption, and wastage all change the same numbers.

1. The problem

Most hotels don't fail because of bad service. They fail because nobody noticed the kitchen was bleeding money for months.

Raw material mismanagement is the silent killer of hotel food-and-beverage. It never shows up as one catastrophic event — it accumulates in a hundred small failures: ingredients ordered without checking the walk-in, stock that expires buried behind a newer delivery, a supplier who short-delivers unnoticed, a staple that runs out mid-dinner, consumption that was never tracked. At month end the food-cost percentage is too high and no one can say exactly why.

Today this is handled by an inconsistent spreadsheet, a paper register nobody trusts, or an expensive ERP that needs a dedicated operator and a consultant. There is no tool built for the people actually doing the work — the chef checking stock before writing tomorrow's menu, the store keeper receiving a delivery at 7am, the manager who needs to know if this week's purchases are on budget before Friday.

2. Who you are building for

Four users, four contexts. Each surface you build serves one of them, and the system's value is the connections between them.

User	Role	Context that shapes the build
Ramesh	Head chef	Carries par levels, yields, and suppliers in his head. When he's on leave the kitchen over-orders by 30%. Wants his knowledge captured so the team can use it without him explaining each time.
Kavitha	Store keeper	Receives deliveries and keeps a handwritten register only she fully understands. Wants her work legible to others without more effort. Often the only person in the building at 6am.
Arjun	F&B manager	Owns food cost, vendors, and approvals. Always surprised by month-end variances he can't trace. Needs to see a problem while it is still a problem, not after the loss.

User	Role	Context that shapes the build
Meena	Purchase manager (chain)	Procures across four properties on last month's invoices, with no real-time view of consumption. Always either over-supplying or hearing something ran out.

Four primary user roles, each working in a different context. There may be multiple users performing the same role across different properties and outlets. For example, a hotel chain may have several store keepers managing inventory at different properties, multiple chefs working across kitchens, and different managers overseeing operations. Each surface should support concurrent users while ensuring that data remains accurate and synchronized across the system.

3. What you are building

One raw-material management system with four connected surfaces over the same live stock data. These are not independent tools — a change on one is visible to the others.

Surface	Who uses it	Target form factor
Store app	Store keeper — receive, issue, adjust stock	Mobile / tablet
Chef view	Head chef — check stock, plan menu, flag needs	Mobile
Manager dashboard	F&B manager — cost tracking, alerts, approvals	Web / phone
Purchase console	Purchase manager — vendor orders, chain-wide view	Web

Build and demo on your laptop. The form factor tells you the layout and interaction model to design each surface for — simulate mobile or tablet by resizing a browser window. It is not the machine you code on. (Responsiveness across devices)

The connections that tie them together:

- When the store keeper receives a delivery and updates stock, the manager's food-cost projection reflects it.
- When the chef flags an ingredient as running low, the purchase manager sees it before service is affected.
- It must work for one property and equally for a chain of four — each property's stock isolated, but a chain manager seeing across all of them.

4. The core engineering challenge

Everything reduces to one technical problem: a single stock quantity per ingredient that several transaction types all mutate, and that must stay correct and reconcilable. As you build, keep asking three questions.

1. **How is a stock balance derived?** Model stock as the sum of an immutable transaction ledger (receive, issue, consume, waste, adjust) rather than a single mutable number — so any balance can be explained and audited.
2. **How do units stay consistent?** Ingredients are purchased in cases or kilograms but consumed in grams or pieces. Conversions must live in the model, not be the user's problem, and must never lose precision.
3. **How is a property isolated?** Scope stock, transactions, and par levels per property so one property can never read or write another's, while a chain manager can aggregate across all of them.

5. Functional requirements

Build the path through a stock lifecycle end to end. Each flow below is something the system must do.

5.1 Receiving a delivery (Store app)

- Record what arrived, flag short or damaged quantities, and update stock — on a phone, in under 3 minutes.
- Associate the delivery against a purchase order or request.
- Handle a partial delivery gracefully (received less than ordered).

5.2 Issuing to the kitchen (Store app)

- Record stock moving from store to kitchen as a distinct transaction.
- Make issue recording fast enough that a busy person actually does it.
- Support manual issue-slip entry (some hotels require it for audit) without breaking the data model.

5.3 Recipe-based consumption (Chef view)

- Link menu items to their raw-material recipes.
- When a dish is marked prepared, deduct the recipe's ingredients from stock — automatically if linked to a POS, or manually logged at end of service.

5.4 Low-stock alerts

- Every ingredient has a par level. Surface approaching and breached par levels.
- Alerts carry context: not just “chicken is low,” but “chicken runs out before Friday service at this week's usage rate.”
- Define who is notified and how they act on it.

5.5 Purchase request & approval (cross-cutting)

- Chef or store keeper raises a request; F&B manager approves or adjusts; purchase manager converts it to a vendor order.
- Build the handoffs between roles explicitly — this is where requests currently get lost.

5.6 Month-end reconciliation (Manager dashboard)

- Compare theoretical consumption (what was sold) against actual consumption (what left the store).
- Surface the variance and let it be explained — without exporting to a spreadsheet.

5.7 Wastage & spoilage

- Record wastage and spoilage as a distinct transaction type, separate from normal consumption, because cost reporting must distinguish the two.

5.8 Stretch — if time allows

- Full automatic recipe deduction wired to a mock POS “dish prepared” event.

6. Constraints

- Store app works with minimal connectivity (basements, cold rooms) — capture offline, sync on reconnect.
- Unit conversions (case ↔ kg ↔ g ↔ piece) handled in the system, never pushed onto the user.
- Vendor management is out of scope, but vendor name, lead time, and price should be storable per ingredient for a future module.
- Manual issue-slip entry must be accommodated without breaking the data model.
- Wastage/spoilage is a separate transaction type from consumption.

7. Out of scope today

Do not spend time on these — they are excluded so you can focus.

- Supplier onboarding and vendor-management portal
- Automatic PO generation and emailing to vendors
- Financial accounting integration (Tally, SAP)
- Barcode / RFID scanning — assume manual quantity entry
- Liquor and beverage management (different compliance)
- Real vendor or payment integration — mock anything external

8. What success looks like (your demo)

By the end of the session, be ready to demonstrate live:

4. A store keeper receives a delivery, logs quantities, and flags a short delivery — in under 3 minutes on an application.
5. A chef checks current stock for a specific ingredient and raises a low-stock request.
6. An F&B manager sees a real-time food-cost percentage and drills into the category driving a variance.
7. A purchase manager sees stock across two properties and generates a consolidated order.
8. Stretch: a recipe deducts ingredient stock automatically when a dish is marked prepared.

9. Tech choices & ground rules

- **Stack is yours.** Any language, framework, and datastore. Use what you are fastest in — we judge the result and your reasoning, not your toolchain.
- **Full-stack expected.** You own the data model, the API, and at least two of the four surfaces end to end, plus the transaction logic that links them.
- **Mock freely.** Seed data, fake auth, and a stubbed POS event are all fine. Correct stock math matters more than breadth.
- **AI tools allowed.** Use whatever you normally use. Be ready to explain any line we point at.

10. How you will be evaluated

Area	Weight	What we look for
Stock-ledger correctness	30%	Balances stay right as receive, issue, consume, waste, and adjust all act on the same ingredient; every balance is explainable.
Data model & multi-tenancy	25%	Clean ingredient/transaction/recipe/par modelling; unit conversion without precision loss; property isolation with chain aggregation.
Functional coverage	20%	How much of the stock lifecycle actually works end to end.
Code quality & reasoning	15%	Readable structure, sensible boundaries, and clear answers to 'why this way?'
Reconciliation & variance	10%	Theoretical vs actual consumption surfaced honestly; wastage distinguished from consumption.

Three product principles we will weigh your choices against:

- **Capture work at the moment it happens.** Records go wrong because updates are batched or skipped. Every screen should be usable in context — on a dock, in a walk-in, between rushes — not at a desk afterward.
- **Make the invisible visible.** Consumption, waste, short deliveries and spoilage are happening unseen. Surface them without creating a reporting burden.
- **Trust but verify.** A store keeper's count is ground truth, but counts have errors and stock disappears in ways records miss. Design for reconciliation and variance, not just recording.

The goal is not a system that's impressive to demo. It's a system a store keeper would actually use at 6am when a delivery arrives and she's the only person in the building. That's the bar.